

4.1 Weather Station

Overview

In this tutorial, you will master the usage of the DHT11 temperature and humidity sensor and the water level sensor, and be able to accurately collect temperature, humidity, and water level data in the current environment. These data can be used to simulate core monitoring indicators such as temperature, humidity, and precipitation required by a weather station.

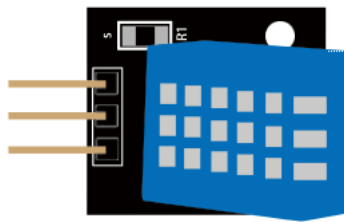
Component Required:

- (1) x Elegoo ESP32
- (2) x 400-hole breadboards
- (1) x DHT11 Temperature and Humidity module
- (1) x water level sensor
- (1) x 0.96 OLED display

Component Introduction

Temp and humidity sensor:

DHT11 digital temperature and humidity sensor is a composite Sensor which contains a calibrated digital signal output of the temperature and humidity. The dedicated digital modules collection technology and the temperature and humidity sensing technology are applied to ensure that the product has high reliability and excellent long-term stability. The sensor includes a resistive moisture sensor and a NTC temperature measurement devices, and connects with a high-performance 8-bit microcontroller.



Applications: HVAC, dehumidifier, testing and inspection equipment, consumer goods, automotive, automatic control, data loggers, weather stations, home appliances, humidity regulator, medical and other humidity measurement and control.

Product parameters

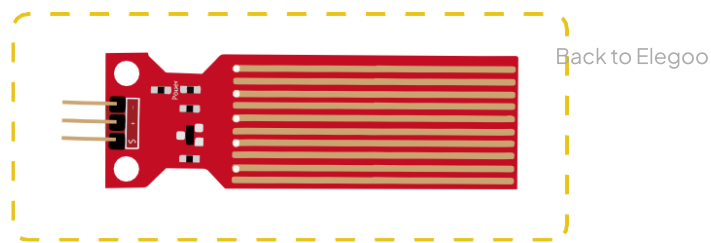
Humidity Parameters		Temperature Parameters	
Parameter	Specification	Parameter	Specification
Relative humidity	1m / s air 6s Hysteresis:	Response time	1 / e (63%) 10S
Resolution	8Bit	Resolution	8Bit
Repeatability	±1% RH	Repeatability	±0.2°C
Accuracy	At 25°C ±5% RH	Range	At 0°C ±50°C
Long-term stability	< ±0.5% RH / yr in	Power supply	DC 3.5~5.5V
Interchangeability	fully interchangeable	Supply Current	measurement 0.3mA standby 60μA
Response time	1 / e (63%) of 25°C 6s	Sampling period	more than 2 seconds

Pin Description:

1. the VDD power supply 3.5~5.5V DC.
2. DATA serial data, a single bus.
3. GND ground, the negative power.

Water Level sensor

A water sensor brick is designed for water detection, which can be widely used in sensing the rainfall, water level, even the liqueate leakage. The brick is mainly composed of three parts: an electronic brick connector, a 1 MΩ resistor, and several lines of bare conducting wires.



This sensor works by having a series of exposed traces connected to ground.

Interlaced between the grounded traces are the sense traces.

The sensor traces have a weak pull-up resistor of 1 MΩ. The resistor will pull the sensor trace value high until a drop of water shorts the sensor trace to the grounded trace. Believe it or not this circuit will work with the digital I/O pins of your UNO/ESP32 board or you can use it with the analog pins to detect the amount of water induced contact between the grounded and sensor traces.

This item can judge the water level through with a series of exposed parallel wires stitch to measure the water droplet/water size. It can easily change the water size to analog signal, and output analog value can directly be used in the program function, then to achieve the function of water level alarm.

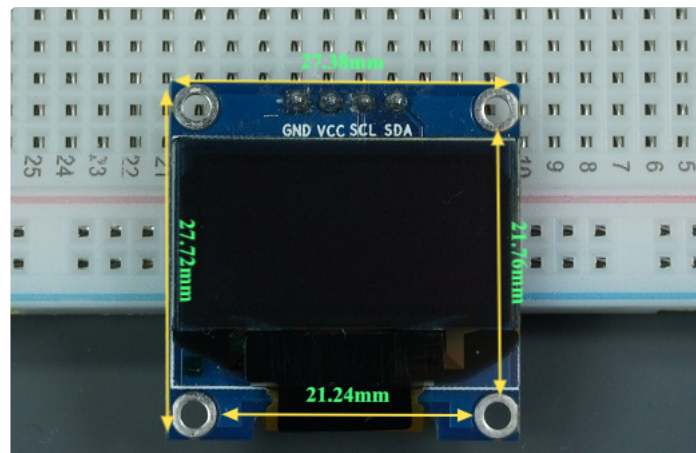
It has low power consumption, and high sensitivity.

Features:

- Working voltage: 5V
- Working Current: <20ma
- Interface: Analog
- Width of detection: 40mm×16mm
- Working Temperature: 10°C~30°C
- Output voltage signal: 0~2.4V

I2C OLED Display

The organic light-emitting diode (OLED) display that we'll use in this tutorial is the SSD1306 model: a monicolor, 0.96-inch display with 128×64 pixels as shown in the following figure.



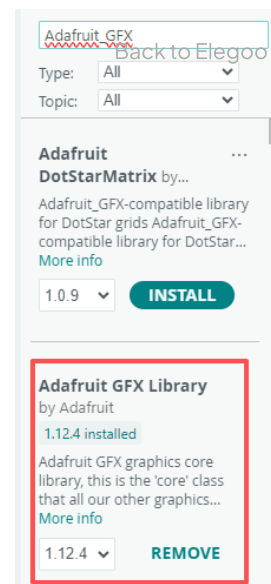
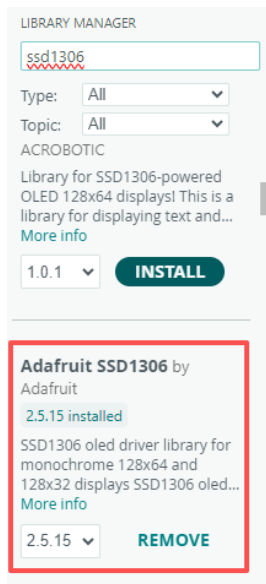
The OLED display doesn't require backlight, which results in a very nice contrast in dark environments. Additionally, its pixels consume energy only when they are on, so the OLED display consumes less power when compared with other displays.

The model we're using here has only four pins and communicates with the ESP32 using I2C communication protocol. There are models that come with an extra RESET pin. There are also other OLED displays that communicate using SPI communication.

Because the OLED display uses I2C communication protocol, wiring is very simple. You just need to connect to the ESP32 I2C pins as D21(SDA), D22(SCL)

To control the OLED display you need the `adafruit_SSD1306.h` and the `adafruit_GFX.h` libraries. Follow the next instructions to install those libraries.

1. Open your Arduino IDE and go to Sketch > Include Library > Manage Libraries. The Library Manager should open.
2. Type "SSD1306" in the search box and install the SSD1306 library from Adafruit.

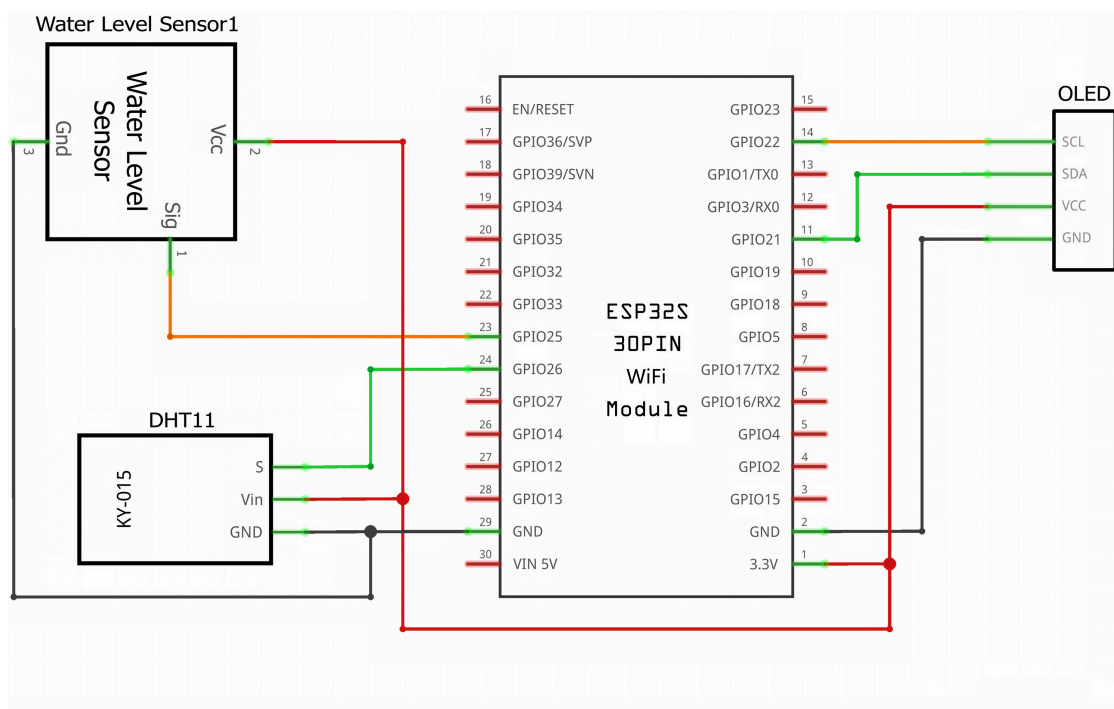


Tips for writing text using these libraries

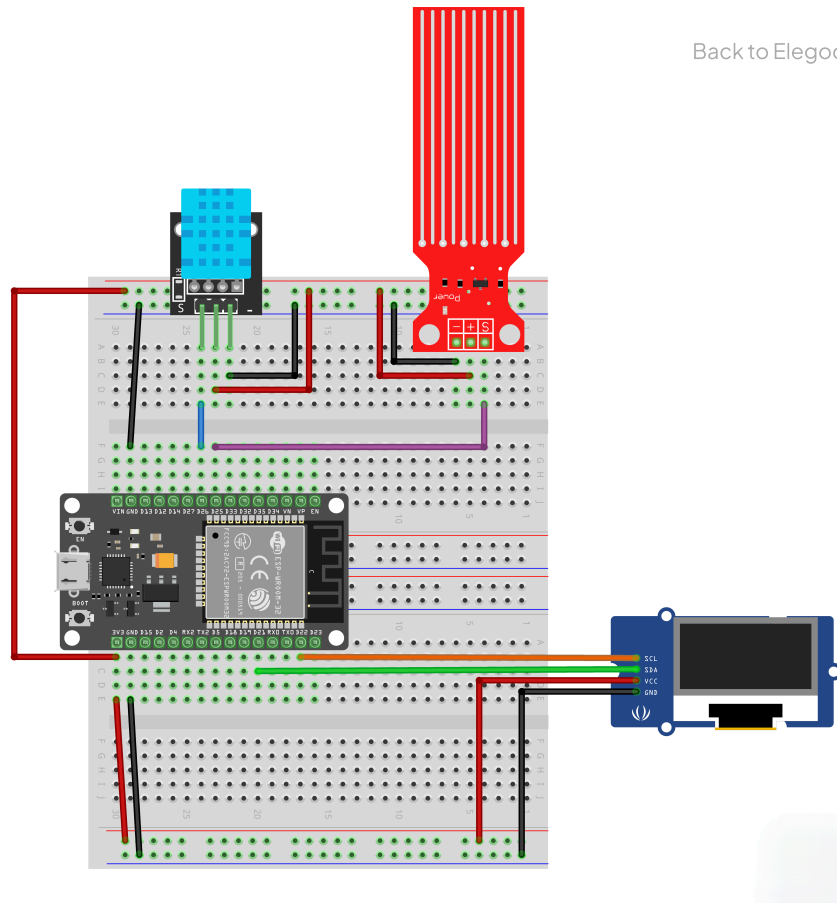
Here's some functions that will help you handle the OLED display library to write text or draw simple graphics.

1. `display.clearDisplay()` – all pixels are off
2. `display.drawPixel(x,y, color)` – plot a pixel in the x,y coordinates
3. `display.setTextSize(n)` – set the font size, supports sizes from 1 to 8
4. `display.setCursor(x,y)` – set the coordinates to start writing text
5. `display.print("message")` – print the characters at location x,y
6. `display.display()` – call this method for the changes to make effect

Connection Schematic



Wiring diagram



Code

After wiring, please open the program in the code folder- weather station and click UPLOAD to upload the program. Before you can run this, make sure that you have installed the library or re-install it, if necessary. Otherwise, your code won't work. This library is stored in the provided folder and needs to be manually imported and added to your project. Please refer to the link below for specific operation methods.

[ide-download](#)

[library-Installation](#)

[FAQS](#)

 [DOWNLOAD weather_station.zip](#)

After opening the program, you will see multiple files, including the .ino main program file, .h header files, and .cpp source files. This program is a weather station data collection and display system, whose core function is to read temperature and humidity data via the DHT11 sensor, collect water level data through the water level sensor, and display all data on the OLED screen in real time. The module call relationship of each file is shown in the figure below.

Note: The water level sensor can work by directly reading the analog pin values, so no separate resource files are required.

```
main (weather_station.ino)
├─ Calls → DHT11 Module
│   └─ DHT11.h (Class/Function/Macro Declarations)
│       └─ DHT11.cpp (Function Implementation: Temp/Humidity Reading)
├─ Calls → OLED Module
│   └─ OLED_Display.h (Class/Function Declarations)
│       └─ OLED_Display.cpp (Function Implementation: Data Display)
```

In the .h header files, there usually include function declarations, definitions of input/output pins, macro definitions, and declarations of relevant data types. By reading the header files, you can quickly grasp the core functions of the corresponding module, the external calling interfaces it provides, and hardware dependencies (such as pin configurations) without checking the specific implementation details.

weather_station.ino DHT11.cpp DHT11.h OLED_Display.cpp OLED_Display.h Back to Elegoo

```

1 // DHT11.h
2 #ifndef DHT11_H
3 #define DHT11_H
4
5 #include <dht_nonblocking.h>
6
7 #define DHT_SENSOR_TYPE DHT_TYPE_11
8 #define DHT_SENSOR_PIN 26
9
10 class DHT_nonblocking_sensor {
11 public:
12     DHT_nonblocking dht_sensor;
13
14     DHT_nonblocking_sensor() : dht_sensor(DHT_SENSOR_PIN, DHT_SENSOR_TYPE) {}
15
16     bool measure(float *temperature, float *humidity);
17 };
18
19 #endif // DHT11_H

```

Similarly, the .cpp source files serve as the implementation carrier for the header files. Their core function is to implement the function logic declared in the .h files, as well as complete variable definitions and initialization. They contain specific code execution processes (such as sensor data reading timing, OLED screen drawing logic, etc.) and are the actual implementers of module functions. Without modifying the header files, you can optimize module performance or adjust functional details only by modifying the code in .cpp files, without affecting the calling interfaces of other modules.

weather_station.ino DHT11.cpp DHT11.h OLED_Display.cpp OLED_Display.h

```

1
2 #include "DHT11.h"
3 #include <Arduino.h>
4
5 bool DHT_nonblocking_sensor::measure(float *temperature, float *humidity) {
6     static unsigned long measurement_timestamp = millis();
7
8     /* Measure once every four seconds. */
9     if (millis() - measurement_timestamp > 3000ul) {
10         if (dht_sensor.measure(temperature, humidity) == true) {
11             measurement_timestamp = millis();
12             return true;
13         }
14     }
15
16     return false;
17 }

```

The OLED_Display.cpp implements the OLED_Display class, leveraging the Adafruit_SSD1306 driver library to provide initialization, data display (temperature/humidity, water level/rainfall), and screen control functions for an OLED display.

weather_station.ino DHT11.cpp DHT11.h OLED_Display.cpp OLED_Display.h

```

1 // OLED_Display.cpp
2 #include "OLED_Display.h"
3 #include <Arduino.h>
4
5
6
7 OLED_Display::OLED_Display() : oled(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET) {}
8
9 void OLED_Display::setupOLED() {
10     Serial.begin(115200);
11     Wire.begin(21, 22); // SDA=21, SCL=22
12     if(!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Address 0x3D for 128x64
13         Serial.println(F("SSD1306 allocation failed"));
14         for(;;);
15     }
16     delay(2000);
17     oled.clearDisplay();
18 }
19
20 void OLED_Display::cleanOled(){
21     oled.clearDisplay();
22 }
23 }

```

Common Pitfalls

1. Member Variable-Function Name Collision:

The OLED instance name must avoid display (which conflicts with the display() refresh function in the Adafruit_SSD1306 library). While the current name oled circumvents this, renaming it to display by mistake will cause compilation errors.

2. Mismatched I2C Pins or Device Address:

The SDA/SCL pins bound in Wire.begin(21, 22) must match the hardware wiring, and the address 0x3C in oled.begin() must be adjusted based on the OLED module model (some use 0x3D). A mismatch will trigger the "SSD1306 allocation failed" error and force the program into an infinite loop.

Since the DHT11 sensor collects data once every 2 seconds and returns valid results only when the reading is successful, special attention must be paid to the persistent display logic of the data: the key is to reasonably plan the calling positions of the screen-clearing function (cleanOled()) and the display refresh function (updatedisplay()). It is recommended to execute the screen-clearing operation at the start of each loop,

and call the unified refresh function only after all data (temperature, humidity, water level) have been written to the display buffer. This process of "clearscreen first → write all data → unified refresh" can avoid screen flickering caused by refreshing before partial data is written, and ensure that historical valid data will not disappear arbitrarily when the sensor reading fails, achieving a stable and persistent display effect.

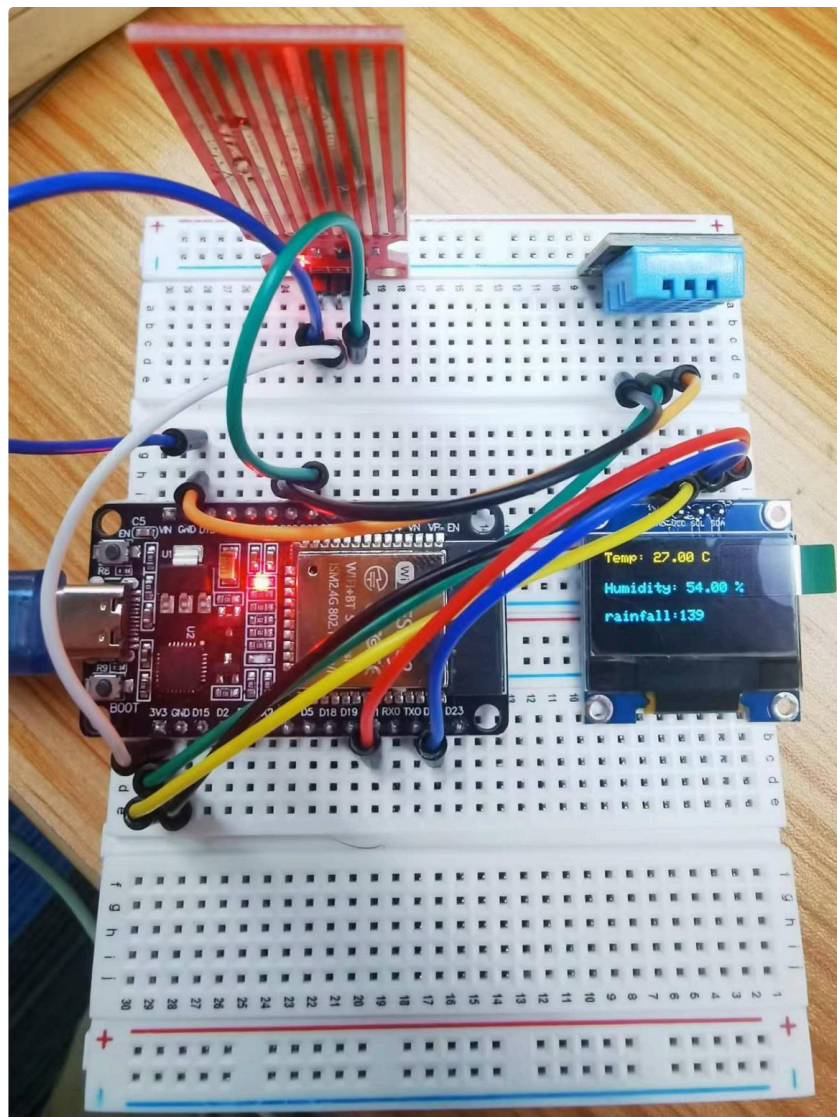
[Back to Elegoo](#)

```
20
21 void loop() {
22
23     // ***** 1. Read DHT every 2 seconds *****
24     if (millis() - lastDHTUpdate > 2000) {
25         float temp, hum;
26
27         if (sensor.measure(&temp, &hum) == true) {
28             temperature = temp; // Update only when successful
29             humidity = hum;
30         }
31         lastDHTUpdate = millis();
32     }
33
34     // ***** 2. Read water level ADC *****
35     int waterLevel = analogRead(adc_id);
36
37     // ***** 3. Refresh OLED (do not clear screen, no flicker) *****
38     display.clearOled(); // Clear the screen once is enough, but if layout changes are needed, put it here as well
39     display.displayTemperatureHumidity(temperature, humidity);
40     display.displayWaterLevel(waterLevel); // Corrected function name from "displayWaterLever" to "displayWaterLevel"
41     display.updateDisplay(); // Corrected function name from "updatedisplay" to "updateDisplay"
42
43     delay(2000);
44 }
```

Upload the program then open the monitor, we can see the data as below:

(It shows the temperature of the environment, we can see it is 27 to 25 degrees with 45.0% humidity)

Click the Serial Monitor button to turn on the serial monitor. The basics about the serial monitor are introduced in details in part 2 Lesson 4.



If you have any questions about the code, please refer to the link below.

<https://wiki.elegoo.com/oshw-getting-started-%26-kits/dht11>

<https://wiki.elegoo.com/oshw-getting-started-%26-kits/watersensor-uno>

<https://wiki.elegoo.com/oshw-getting-started-%26-kits/oled>

